Customer Review Analysis - Project report

Project Overview

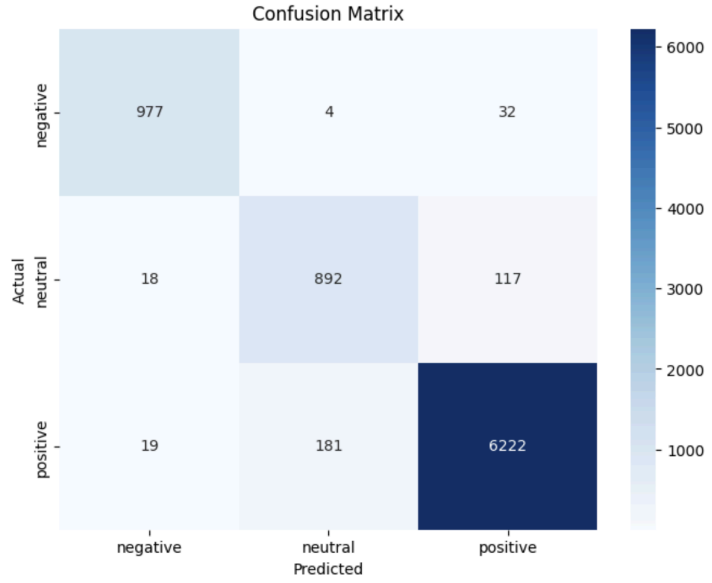
The goal of the project was to create an application that analyses customer reviews, and aggregates them into a dashboard with business insights. The application should have three components: sentiment analysis, product clustering into categories, and summarization (summarizing identified product features into a report or blog post). My project implements an end-to-end NLP pipeline to transform raw, unstructured product data into structured market intelligence.

Technical Approach

Sentiment Analysis
I utilized a fine-tuned RoBERTa model for this task. The model was fine-tuned on an Amazon review dataset of almost 40000 examples. I prepared the data by dropping entries with missing review text, removing duplicates, combining the review title and text into one column, and mapping the rating to a numeric value (to be later interpreted as positive, negative, or neutral). The overwhelming majority of the examples contained positive reviews, so I handled that by oversampling and using a custom weighted trainer.
The model achieved the following results:

Classification Metrics Table:

	precision	recall	f1-score	support
negative	0.963511	0.964462	0.963986	1013.000000
neutral	0.828227	0.868549	0.847909	1027.000000
positive	0.976613	0.968857	0.972719	6422.000000
accuracy	0.956157	0.956157	0.956157	0.956157
macro avg	0.922783	0.933956	0.928205	8462.000000
weighted avg	0.957035	0.956157	0.956526	8462.000000

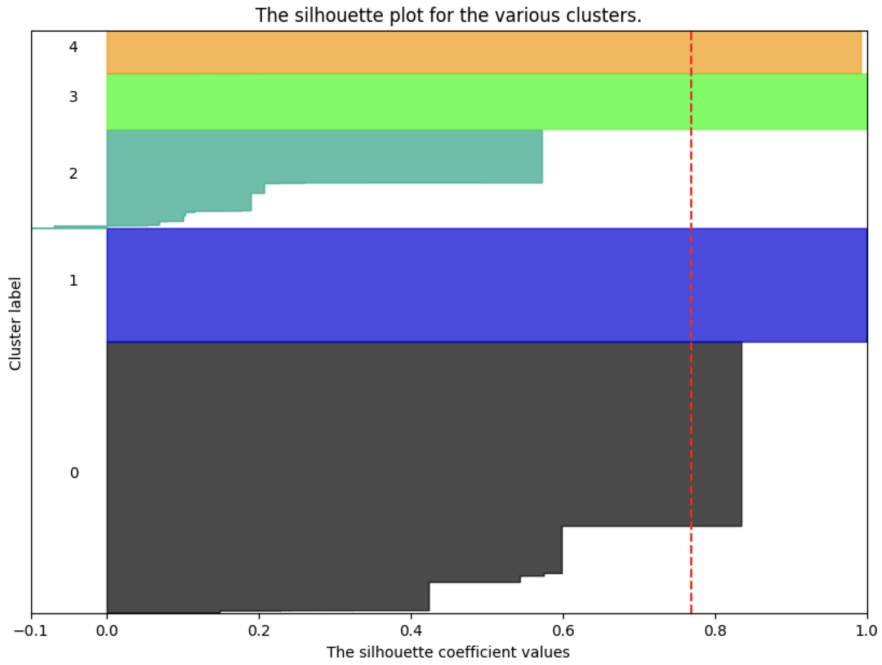


As expected, it struggled most with neutral reviews. This is because neutral reviews most often don't contain strong sentiment, and might even contain mixed sentiment (ex. "decent quality, but arrived late").

Product Category Clustering

I used K-Means clustering to group products into categories. For 5 clusters, I achieved an average silhouette score of 0.77, which confirms that the model identified a strong structure. However, the limitation of this approach is that when uploading a file with real reviews, we don't actually know in advance how many product categories there will be. So as an improvement for the future, we should let the model first identify the optimal number of clusters.

Once I had my clusters mapped back to the original dataframe, I used an OpenAI call (gpt-4o-mini) to generate appropriate labels for the category by sending it a sample of products in that category.



Summarization

For the summarization task, I used the BART-Large-CNN model. First, I identified the best and worst products (overall and in each category), according to the following logic: if the uploaded user file contained ratings (1-5), the best products were those with the highest average rating, with at least 10 reviews. If ratings were not available, I used sentiment instead (products with the highest percentage of positive reviews, at least 10 reviews). The same logic applies to worst products. I gave the model the following prompt to generate product reviews:

```
f"Summarize why '{product_name}' is a {role} in the {category}
category: "
```

The model generated results like the following:

```
============================
MARKET REPORT: OFFICE LIGHTING
============================

### TOP PERFORMING PRODUCTS

**LED Desk Lamp**
Insight: 'LED Desk Lamp' is a top-rated highlight in the Office Lighting category. Perfect brightness settings for night work.
```

The result is not as natural and smooth as we would expect for a blog post, but for the purposes of a report where we need to identify the qualities of the best- and worst-performing products, it works.

Application

As a final deliverable, I created an application which allows the user to upload a CSV file with reviews, and analyzes it using my ML pipeline. I used Flask for the backend, HTML/CSS/JS for the frontend, and Chart.js for visualization. I implemented asynchronous data processing to improve user experience and not show a frozen page while the data is being processed, as well as to prevent requests timeouts. The biggest challenge while building the app was proper data handling and anticipating edge cases: user uploads data with invalid columns, or missing data, or incorrect data format, or the number of reviews is not enough to run the analysis, etc.